

When the amount of data increases, a large coefficient for a smaller order won't make a difference compared to the higher orders. At a certain size  $n$ , the higher orders will always be slower than the lower orders. For example, let's say we have `quadraticExample()`, which is  $O(n^2)$  and has  $3n^2$  steps. We also have `linearExample()`, which is  $O(n)$  and has  $1,000n$  steps. It doesn't matter that the 1,000 coefficient is larger than the 3 coefficient; as  $n$  increases, eventually an  $O(n^2)$  quadratic operation will become slower than an  $O(n)$  linear operation. The actual code doesn't matter, but we can think of it as something like this:

---

```
def quadraticExample(someData): # n is the size of someData
    for i in someData:         # n steps
        for j in someData:     # n steps
            print('Something') # 1 step
            print('Something') # 1 step
            print('Something') # 1 step

def linearExample(someData):   # n is the size of someData
    for i in someData:         # n steps
        for k in range(1000):  # 1 * 1000 steps
            print('Something') # (Already counted)
```

---

The `linearExample()` function has a large coefficient (1,000) compared to the coefficient (3) of `quadraticExample()`. If the size of the input  $n$  is 10, the  $O(n^2)$  function appears faster with only 300 steps compared to the  $O(n)$  function with 10,000 steps.

But big O notation is chiefly concerned with the algorithm's performance as the workload scales up. When  $n$  reaches the size of 334 or greater, the `quadraticExample()` function will always be slower than the `linearExample()` function. Even if `linearExample()` were  $1,000,000n$  steps, the `quadraticExample()` function would still become slower once  $n$  reached 333,334. At some point, an  $O(n^2)$  operation always becomes slower than an  $O(n)$  or lower operation. To see how, look at the big O graph shown in [Figure 13-3](#). This graph features all the major big O notation orders. The x-axis is  $n$ , the size of the data, and the y-axis is the runtime needed to carry out the operation.